

Understanding and Utilising Functions in Programming

Mindaugas Taujanskas

Abstract—Functions are the fundamental building blocks of any software application. They split the software into naturally bite-sized chunks, and simultaneously lend the power of modularity, reusability, and maintainability. A software developer can shape how their program speaks to a reader by the appropriate categorisation and naming of their functions.

I. MODULARITY AND CODE ORGANISATION

Modularity means that functions enable their own reusability across different areas of the program, and are not necessarily one-shot functions intended to simply separate a larger procedure into smaller chunks—although this should be the goal of refactoring. Consider the following excerpt:

```
def create_service_provider_detail_form(self)
    -> RoleSelectionForm:
    body_widget = Form()
    body_layout = QVBoxLayout(body_widget)

    def add_field_row(*fields: QWidget,
                     stretch: bool = True):
        row = QWidget()
        layout = QHBoxLayout(row)
        layout.setSpacing(15)
        for field in fields:
            layout.addWidget(
                field,
                stretch=1 if stretch else 0
            )
        body_widget.fields.append(field)
        body_layout.addWidget(row)

    add_field_row(
        LineEdit("Address Line 1", name="address-1"),
        LineEdit("Line 2", name="address-2")\
            .set_optional(True),
        LineEdit("Post code", name="post-code")
    )

    add_field_row(
        ComboBox("Country", name="country"),
        LineEdit("Home telephone",
                name="home-telephone")
    )

    # ...
```

Without necessary knowledge of the underlying GUI library, it should appear obvious what the mechanism behind the function `create_service_provider_detail_form` inhabits, and it should allow the same layman to modify existing fields, or extend them. The modularity lies within the nested function `add_field_row`, which concisely encapsulates the gritty boilerplate code necessary to append a field to the current GUI layout, and parameterises properties like whether the field should stretch horizontally.

II. REUSABILITY

Reusability rests on the same branch as modularity, as it is itself a sub-requirement of modularity. When writing code, it will greatly benefit a developer to see briefly into the future and wonder if they will need to imitate the same behaviour, and thus whether it should be extracted into a reusable component, whether that is a function, class, interface, etc.

Consider the following (reduced) code:

```
class StyledWidget:
    PRIMARY_COLOR = "#AAAAAA"
    FOCUS_COLOR = "#89a69f"
    ERROR_COLOR = "#F67279"

    def setup_label(self, label: str) -> None:
        """Create a small caption above the widget"""

    def set_label_focus_style(
        self,
        label: QLabel,
        focused: bool
    ) -> None:
        """Set how the label appears when the widget is
        focused"""

    def set_state(self, state: str = "") -> None:
        """Set whether the widget has a valid input"""

    def apply_styling(
        self,
        *,
        stylesheet: str = "",
        padding: str = "24px 4px 4px 4px",
        font_size: int = 12
    ) -> None:
        """Customise the widget's appearance"""
```

This class and its methods enable the reusability and consistency of this base widget style across any UI component the software developer may decide to implement, whether it be a line-edit, combo-box, date picker, etc.. The consequence of this design choice means that the software developer can ensure they avoid having inconsistent styling across the application, less time is spent on debugging tedious mistakes which can arise when dealing with tricky widget styling, and then if the developer decides they want to redesign the UI, they can modify the reused component and have it quickly propagate.

III. READABILITY AND MAINTAINABILITY

Code should be made readable for yourself and for others, both at the time of conception, and when you inevitably need to maintain it a month down the line. Good practices are developed proportionally through time spent writing code, and when having to endure looking back on your or other

developer's code—but writing maintainable code is a far more complicated endeavour which pulls from many facets of the developer's expertise. Maintainability means not only that the code should be readable, but also easily extended upon and diagnosed relative to the complexity involved. If a simple issue arises, the diagnosis should be simple, and the solution should not have negatively cascading effects. Ensuring maintainability is not simply adhering to a set of good practices, it is something that should be rigorously implemented into the design phases, and only then further supplemented by the addition of good practices.

Consider the project folder structure (simplified) of my application:

```
removals_system/
├── assets/
├── components/
│   ├── forms/
│   │   ├── login.py
│   │   └── ...
│   ├── primary_button.py
│   └── styled_widget.py
├── exceptions/
│   └── auth_exceptions.py
├── config/
│   ├── constants.py
│   └── settings.py
├── models/
│   ├── user.py
│   └── db.py
├── views/
│   ├── authentication.py
│   └── dashboard.py
└── controllers/
    ├── authentication.py
    └── dashboard.py
```

Suppose a developer wants to style the login button differently depending on the time of day—assuming the developer recognises the model-view-controller pattern—they should immediately be looking at `controllers/authentication.py`, where the login form's logic must be implemented, and furthermore considering if changes to `components/primary_button.py`, or the higher-level `components/styled_widget.py` would be appropriate to better support these kinds of changes in future.

This structure is maintainable, standardised, and open-ended. It compliments the UI development mindset, and is intuitive for developers that intend to work on the project.

IV. TESTING AND DEBUGGING

Software—especially customer-facing software—must act verifiably predictably, and it is unanimously accepted that testing must exist to prove this fact. The developer's initial step towards testing their software is first and foremost writing software which can be tested. This means that the code must be at least partially compartmentalised into isolated units (e.g., functions, classes, larger functional units,) which produce repeatable outputs with the same inputs.

Consider the project hierarchy from before, the architecture inherently promotes decoupling different functional aspects of a project into smaller pieces, which then paves an easier road from which a developer can write simpler isolated, testable, functional units. Note that with UI development, testing individual UI components tends to be meaningless in a vacuum, but testing to ensure they exhibit the correct UI flow (i.e. login button should submit the form, and fail if the details are invalid, otherwise move to the next view.)

For example, consider entering a bid onto an order as a service provider: there are various constraints that you cannot violate, both voluntarily, and neither a result of poor programming:

```
CREATE FUNCTION bid_modify()
BEGIN
    -- Basic validation
    IF NOT EXISTS (/* order with matching id */) THEN
        RAISE EXCEPTION 'Order % does not exist', NEW.
            order_id;
    END IF;

    IF EXISTS (/* accepted bid for this order */) THEN
        RAISE EXCEPTION 'Cannot modify finalized order';
    END IF;

    -- Handle action types
    IF NEW.action_type = 'bid' THEN
        -- Customer bids must be lower than current
        -- highest
        IF NEW.bidder_id = (/* order customer */) THEN
            IF NEW.bid_amount >= (/* current highest bid
                */) THEN
                RAISE EXCEPTION 'Customer bid too high';
            END IF;
        -- Provider bids must be higher
        ELSE
            IF NEW.bid_amount <= (/* current highest bid
                */) THEN
                RAISE EXCEPTION 'Provider bid too low';
            END IF;
        END IF;
    END IF;
```

```

ELSIF NEW.action_type = 'accept' THEN
    UPDATE BidActions SET bid_status = 'accepted'
    WHERE /* matching active bid */;
    RETURN NULL;

ELSIF NEW.action_type = 'withdraw' THEN
    UPDATE BidActions SET bid_status = 'withdrawn'
    WHERE /* user's active bid with amount */;
    RETURN NULL;
END IF;

RETURN NEW;
END;

```

Although the code is heavily simplified for the sake of clarity, these business-level constraints must be tested thoroughly by mocking [creating pretend] orders to verify that the system operates properly under a well-functioning environment, and conversely prevents incoherent data from being entered if a poorly-programmed interface attempts to create inconsistent bids. A system should adhere strictly to its business requirements, and be designed to prevent actors skirting around them.